

Smaky 6 — 3 · MATPAC : appels arithmétiques et de gestion de fichiers

Transcription du document EPSITEC-system sa, *Février 1982*. Original scanné mis à disposition par Jean-Daniel Nicoud (mars 2024), numérisé par micromusee.ch (Smaky 6 - 3 MATPAC.pdf, 18 pages). Transcrit en Markdown depuis les pages scannées. Le contenu technique (conventions de registres, codes d'erreur, valeurs octales) a été reporté fidèlement ; quelques tokens de listing restent signalés [?] là où le scan est ambigu — à recouper avec l'original en cas de doute.

Mode d'emploi de MATPAC

Généralités

Ce *package* est construit sous forme d'**extension des appels du système**. Il réside en mémoire depuis l'adresse **25000** et occupe **11400 bytes**.

Il comprend deux parties :

- **Appels mathématiques**
- **Appels de gestion de fichier à structure de record**

Comment utiliser MATPAC

Pour pouvoir utiliser les appels de ce *package*, il faut évidemment que celui-ci soit chargé en mémoire. Pour cela il suffit de mettre dans son programme l'instruction suivante :

```
.REF      MAT
```

Le `.REF` donne accès aux symboles du *package*.

Pour charger le *package*, l'utilisateur adjoindra **au début de son programme** les instructions suivantes, qui se chargent d'initialiser l'adresse de l'extension de la table des numéros d'appel `TABEX` et de charger le *package* :

```
ERNSA      =      34                ; message "no starting address"

LOAD       HL, #VATABEX              ; initialise l'adresse de
LOAD       TABEX, HL                 ; l'extension de la table
LOAD       DE, #TMATH                ; pointe le nom du package
.W         ?LGO                      ; charge le package
COMP       A, #ERNSA                 ; teste si le chargement est
JUMP       NE, ERROR                 ; correct, sinon traitement de l'erreur
                                           ; au gré de l'utilisateur
```

```
TMATH:     .ASCIZ    /MATPAC.SY/
```

Cette méthode est la plus sûre, puisqu'elle assure un chargement au début du *package*.

Mais un utilisateur qui emploierait systématiquement les appels du *package* **et qui est sûr de ne jamais détruire la partie mémoire occupée par le *package* ou de modifier TABEX**, peut utiliser la méthode suivante qui charge automatiquement le *package* au moment du RESET.

Il suffit d'assembler le petit programme suivant et de l'appeler $ST.SY$; ainsi il sera exécuté à chaque reset :

```
.TITLE    MATPAC  LOADER
.PROC     Z80
.REF      FLO

.LOC      53000

ERNSA     =      34                ; message "no starting address"

START:
LOAD      HL,#VATABEX              ; initialise l'adresse de
LOAD      TABEX,HL                 ; l'extension de la table
START1:
LOAD      DE,#TMATH                ; pointe le nom du package
.W        ?LGO                     ; charge le package
COMP      A,#ERNSA                 ; teste si le chargement est
JUMP      NE,ERROR                 ; correct, sinon erreur
.W        ?TEXTIM
.ASCIIZ   /<CR>MATPAC LOADED/
.W        ?RTN                     ; retour au CLI

ERROR:
.W        ?BUZZ,?BUZZ,?BUZZ
.W        ?TEXTIM
.ASCIIZ   /<CR>MATPAC LOAD ERROR: /
.W        ?ERROR
.W        ?TEXTIM
.ASCIIZ   /<CR>TRY AGAIN ? /
.W        ?GETCAR,?DICAR
CLR       A:5
COMP      A,#'Y
JUMP      EQ,START1
.W        ?RTN

TMATH:    .ASCIIZ   /MATPAC.SY/

.END      START
```

Pour le reste, l'utilisation des appels est exactement semblable aux appels du système.

Appels mathématiques

Ces appels sont réalisés autour de routines de calcul **ZILOG** écrites pour le processeur Z80.

Ces appels permettent de traiter des **nombre réels de 13 chiffres significatifs** allant de 10^{-127} à 10^{+126} .

L'affichage des nombres peut se faire en trois modes différents :

- virgule flottante
- virgule fixe
- notation scientifique

Caractéristiques des variables binaires et ASCII

Variables binaires (codées décimales)

Les variables binaires sont en **binaire codé décimal (BCD)** et nécessitent **8 bytes**. Format :

octet 0	octets 1 ... 6	octet 7
signe S X X X	mantisse : 13 digits BCD (di-digit)	exposant

- **signe** (S:X:X:X) — S = bit de signe (⁰ = positif) ; X = bits non significatifs.
- **mantisse** — 13 chiffres BCD.
- **exposant** — biaisé (valeurs en **octal**) :

exposant (octal)	valeur
177	10^{-2}
200	10^{-1}
201	10^0
202	10^1

Variables ASCII en entrée

Les variables ASCII peuvent avoir plus de 13 chiffres significatifs. Cependant, seuls les 13 premiers chiffres seront pris en compte. **Par contre, l'exposant tiendra compte de tous les chiffres présents.** Les caractères espace sont ignorés.

Les nombres peuvent être exprimés en notation scientifique. On utilise à cet effet la lettre ^E majuscule. La fin de la variable ASCII n'est pas marquée par un signe particulier mais par le premier caractère hors syntaxe.

Syntaxe :

- Le premier caractère peut être un signe moins, un signe plus ou un chiffre.
- Les caractères suivants des chiffres, un point ou la lettre ^E majuscule pour signifier l'entrée d'un exposant.
- On ne peut avoir qu'un seul point et qu'une seule lettre ^E.
- Après la lettre ^E on peut avoir soit un chiffre soit le signe moins, le signe plus puis d'autres chiffres.
- Ne sont considérés comme tapés que les caractères visibles sur l'écran. On peut donc par exemple retaper la lettre ^E si la précédente lettre ^E a été effacée par BS ou DEL.

Exemples de chaînes ASCII :

```
123456789098765432.123456*
└── pris en compte ─┘ L "*" : pris comme terminateur
    (ignoré au-delà de 13 chiffres significatifs)
```

```
+1223.234 E-12=+234.689E-13x
└ pris en compte ┘ L début 2ème chaîne ; "x" : terminateur
```

Quelques autres chaînes acceptées :

```
.ASCIIZ  /-.345E-2/  
.ASCIIZ  /00000.0000001/  
.ASCIIZ  /12.96E-38 = coefficient de correction/
```

Quelques chaînes inacceptables :

```
.ASCIIZ  /E-14/           exposant seul  
.ASCIIZ  /12.1234E-A1/     lettre dans l'exposant  
.ASCIIZ  /12346.1234 EABC/ idem
```

Variables ASCII en sortie

Les appels `?BINDEC`, `?DIBIDE`, `?WRBIDE` transforment une variable binaire en une variable ASCII.

Le format de cette variable est soit une **notation en virgule fixe**, soit une **notation en virgule flottante**, soit une **notation scientifique**.

Pour la notation en virgule fixe, le nombre de digits (chiffres avant la virgule) ainsi que le nombre de décimales (chiffres après la virgule) peut être sélectionné par l'appel `?DECFOR`. Lorsqu'il n'est pas possible d'afficher la variable selon le format sélectionné, ces appels passent automatiquement en notation scientifique avec le maximum de chiffres significatifs. Ce passage se fait soit si l'ordre de grandeur du nombre à afficher est supérieur au nombre de digits sélectionné ou si les nombres plus petits que 1, si le premier chiffre significatif n'est pas affichable avec le nombre de décimales sélectionné.

On peut également forcer dans tous les cas la notation scientifique ou sélectionner la notation virgule flottante à l'aide de cet appel.

`?DIBIDE` et `?WRBIDE` cadrent le nombre, c'est-à-dire que le nombre de positions qui sépare le pointeur d'origine de la position des unités est constant et correspond au nombre de digits sélectionnés plus une position pour le signe. Les positions inutilisées sont remplies avec des espaces. Par contre si l'on veut par exemple afficher un nombre directement après un texte, les positions inutilisées remplies d'espaces sont gênantes. C'est la raison pour laquelle l'appel `?WRBIDE` affiche depuis le premier chiffre ou signe.

nombre cadré	nombre non cadré
12.2350	12.2350
-123452.2340	-123452.2340
0.0012	0.0012
1235445.5462	1235445.5462
192.2390	192.2390
-1.0000	-1.0000

Liste des erreurs (appels mathématiques)

Symbole	Code	Signification
ERLLI	5	line too long
ERZDI	37	divide by zero
EROVE	40	overflow
ERUND	41	underflow
ERILN	42	illegal number
ERSQR	43	negative square-root

Description des appels mathématiques

Convention générale : une variable flottante est une **cellule de 8 octets** en mémoire ; on la désigne par un **pointeur** dans ^{HL} et/ou ^{DE}. Sauf indication contraire, les résultats des opérations remplacent l'opérande pointé par ^{HL}. En cas d'erreur, retour **CARRY SET** avec le numéro d'erreur dans ^A.

Conversions et entrées/sorties

Appel	Effet	Entrée	Sortie	Modifie	Erreurs
?BINDEC	binaire → chaîne ASCII	^{HL} → binaire	^{DE} → ASCII (mémoire)	rien	—
?WRBIDE	binaire → écran (non cadré, via ?DICAR)	^{HL} → binaire	écran	rien	—
?DIBIDE	binaire → écran (cadré)	^{HL} → binaire	écran	rien	—
?DECBIN	chaîne ASCII → binaire	^{HL} → ASCII	^{DE} → binaire	AF	40, 41, 42
?HLTBIN	entier 16 bits signé → binaire BCD	^{HL} = entier	(^{DE}) = binaire	rien	—
?BINTHL	binaire BCD → entier 16 bits	(^{DE}) = binaire	^{HL} = entier	AF	dépassement sup./inf. [?]
?GETDEB	saisie clavier (avec écho) → binaire	clavier	(^{DE}) = binaire	AF	40, 41, 42
?DECFOR	règle le format d'affichage	B = nb max de digits, C = nb de décimales	—	BC	—

Notes :

- ?DECFOR : on peut sélectionner au maximum **13 digits** et **12 décimales**. Si le nombre de décimales dépasse les digits, on a un cas respecté des règles : l'appel corrige automatiquement. Si l'on spécifie 0 digit et un nombre de décimales non nul, on force l'affichage permanent en notation scientifique. Si l'on spécifie 0 digit et 0 décimale, on sélectionne l'affichage en virgule flottante.
- ?GETDEB : la touche ^{DEL} permet de revenir jusqu'au début de la saisie ; la saisie se termine au premier caractère hors syntaxe.

Opérations binaires (deux opérandes)

Appel	Effet	Modifie	Erreurs
	$(HL) = (HL) \text{ op } (DE), \text{ op choisie par le caractère ASCII dans } ^A (+ - * /), \text{ défaut} = +$		
?FOPER		AF	—
?FADD	$(HL) = (HL) + (DE)$	AF	40, 41
?FSUB	$(HL) = (HL) - (DE)$	AF	40, 41
?FMUL	$(HL) = (HL) * (DE)$	AF, DE	40, 41
?FDIV	$(HL) = (HL) / (DE)$	AF	37, 40, 41

?FPOW (HL) = (HL) puissance (DE) AF 40, 41

Fonctions unaires (un opérande, résultat à la place de (HL))

Appel	Effet	Modifie	Erreurs
?FSQRT	(HL) = racine carrée (HL)	AF	43
?FEXP	(HL) = exponentielle (HL)	AF	40, 41
?FLOG	(HL) = logarithme népérien (HL)	AF	44
?FSIN	(HL) = sinus (HL)	rien	—
?FCOS	(HL) = cosinus (HL)	rien	—
?FTAN	(HL) = tangente (HL)	AF	40, 41
?FATAN	(HL) = arc-tangente (HL)	rien	—
?FINVER	(HL) = 1 / (HL)	AF	37
?FRANDOM	(HL) = nombre aléatoire entre 0 et 1	rien	—
?FINT	(HL) = partie entière (HL)	rien	—
?FFRAC	(HL) = partie fractionnaire (HL)	rien	—
?FABS	(HL) = valeur absolue (HL)	rien	—
?FCHSIG	(HL) = signe opposé (HL)	rien	—
?FRND	(HL) = arrondi (HL) sur la dernière	rien	—
	décimale (nb de décimales par ?DECFOR)		
?FRND5	(HL) = arrondi à 5 sur la dernière	rien	—
	décimale		

Exemple d'arrondi ?FRND5 effectué pour 2 décimales :

1.92 => 1.90
1.93 => 1.95
1.97 => 1.95
1.98 => 2.00

Constantes

Appel	Effet	Modifie
?FCLR	(HL) = 0	rien
?FPI	(HL) = π	rien

Comparaison

?FCOMP — F = COMP (HL) , (DE) . Effectue la comparaison entre la variable pointée par HL et celle pointée par DE. Les bits du registre F sont mis à jour et permettent d'utiliser les conditions suivantes :

Condition	Drapeaux
higher or same ($\geq$)	carry clear
lower ($<$)	carry set
non equal ($\neq$)	zero clear
equal ($=$)	zero set

Appels de gestion de fichier à structure de record

Définitions

L'utilisation de ces appels permet la création et la gestion de fichier disque ayant la structure suivante :



En-tête du fichier I:R:F:E:Z :

- **I** — byte d'identification de fichier record (code **371**)
- **R** — longueur d'un record, de 1 à 255 bytes max
- **F** — numéro du prochain record libre
- **E** — nombre de records dans le fichier

Le fichier contient une **en-tête** qui décrit le fichier, une **partie utilisée** et une **partie vide**. Ces deux parties sont subdivisées en records de longueur égale et définie à la création du fichier. La frontière entre ces deux parties représente la fin du fichier. Chacune de ces parties peut évidemment être de longueur nulle. Ainsi, immédiatement après sa création, le fichier ne contiendra qu'une partie vide. L'appel ?ADDREC permet de créer, puis d'agrandir la partie utilisée. Les autres appels traitant des records se rapportent à la partie utilisée exclusivement.

Généralités et recommandations

Ces appels sont constitués à l'aide des appels de base de SAMOS. Il est donc nécessaire de bien avoir présent à l'esprit le fonctionnement des appels de SAMOS et de se référer si besoin est à la notice SAMOS.

Les fichiers à structure de records seront exclusivement traités par les appels de ce *package*. **On ne peut pas, par exemple, ouvrir le fichier avec un appel SAMOS puis le traiter avec des appels de ce package.**

Il est par contre possible d'utiliser séparément mais simultanément des fichiers simples avec directement les appels SAMOS, et des fichiers à structure de records avec les appels de ce *package*. **La seule restriction importante est de ne pas utiliser l'appel SAMOS ?RESET tant que des fichiers à structure de records sont ouverts.**

Les limites

Les limites globales sont naturellement celles de SAMOS. Cela vaut surtout si l'on traite simultanément les deux types de fichier. Voici cependant les limites spécifiques du *package* valable si l'on ne traite que des fichiers à structure de records :

- au maximum **4 fichiers ouverts simultanément** ;
- longueur du record comprise entre 1 et **255 caractères** ;
- longueur minimum d'un buffer de travail égale à **2 blocs**.

Les appels du *package* n'utilisent que l'ouverture en écriture en accès par blocs ; durant l'exécution de l'appel `?RCREAT`, on dispose donc toujours avec SAMOS, de **8 canaux en écriture**. Le *package* n'utilise pas les buffers de SAMOS.

Description des appels de gestion de fichier

`?RCREAT` — crée un fichier à structure de records

Création d'un fichier avec réservation de la bonne taille calculée à partir de la longueur du record et du nombre de records, écriture de tous les blocs avec dans le bloc 0 l'en-tête, fermeture du fichier.

- **Entrée** : DE = pointeur au nom ; BC = nombre de records ; A = longueur du record.
- **Sortie** : CC = exécution correcte ; CS = erreur, numéro dans A .
- **Modifie** : AF .

Code	Erreur	Code	Erreur
11	file already exist	30	device timeout
13	illegal filename	31	write protect tab set
21	unknown device	32	write error
24	all channel in use	33	read error
25	directory full	45	record length nul
26	disk full	46	zero record

`?ROPEN` — ouvre un fichier à structure de records

Ouvre un fichier pour lecture, écriture ou addition de records. L'ouverture du fichier est faite en accès par bloc. Le buffer de travail commence depuis DE , longueur $\geq$ **2 blocs minimum** (le buffer ne doit pas être situé dans une zone absolument libre). Le pointeur du fichier (position dans le fichier) est sur le record 0.

- **Entrée** : DE = pointeur au nom ; HL = pointeur début du buffer ; A = nombre de blocs pour le buffer.
- **Sortie** : CC = correct, no de canal dans A ; CS = erreur, numéro dans A .
- **Modifie** : AF .

Code	Erreur	Code	Erreur
2	read protect file	24	all channel in use
12	file does not exist	30	device timeout
13	illegal filename	32	write error
20	file in use for reading	33	read error
21	unknown device	50	buffer too small

`?RCLOSE` — ferme un fichier à structure de records

S'il y a lieu, la dernière tranche traitée du buffer est écrite sur le fichier (écriture de records) et l'en-tête est mise à jour (addition de records). Finalement le fichier est fermé.

- **Entrée** : A = canal.
- **Sortie** : CC = exécution correcte ; CS = erreur, numéro dans A .
- **Modifie** : AF .
- **Erreurs** : 22 = channel error ; 30 = device timeout ; 32 = write error ; 33 = read error.

?RRESET — ferme tous les fichiers à structure de records ouverts

Cet appel n'a pas de contrôle d'erreur. Si une erreur survient durant la fermeture d'un canal, celui-ci est purement et simplement supprimé. Cet appel ne doit en principe être utilisé que pour une fermeture d'urgence.

- **Modifie** : rien.

?RDREC — lit le record courant

Transfert du buffer à l'utilisateur. Si le record courant n'est pas dans le buffer, écrit s'il y a lieu le buffer dans le fichier, puis lit la tranche du fichier qui contient le record courant. Le pointeur du record courant n'est pas modifié.

- **Entrée** : A = canal ; DE = pointeur en mémoire.
- **Sortie** : CC = exécution correcte ; CS = erreur, numéro dans A .
- **Modifie** : AF .
- **Erreurs** : 22 = channel error ; 30 = device timeout ; 32 = write error ; 33 = read error.

?WRREC — écrit le record courant

Transfert de l'utilisateur au buffer. Si le record courant n'est pas dans le buffer, écrit s'il y a lieu le buffer dans le fichier, puis lit la tranche du fichier contenant le record courant. Mémoire qu'une opération d'écriture a eu lieu et qu'il faudra donc écrire le buffer dans le fichier. Le pointeur du record courant n'est pas modifié.

- **Entrée** : A = canal ; DE = pointeur en mémoire.
- **Sortie** : CC = exécution correcte ; CS = erreur, numéro dans A .
- **Modifie** : AF .
- **Erreurs** : 22 = channel error ; 30 = device timeout ; 32 = write error ; 33 = read error.

?NEXREC — pointe le record suivant

- **Entrée** : A = canal.
- **Sortie** : CC = exécution correcte ; CS = erreur, numéro dans A .
- **Modifie** : AF .
- **Erreurs** : 6 = end of file ; 22 = channel error.

?ADDREC — ajoute un record au fichier

Transfert de l'utilisateur au buffer. Si le record en ajout n'est pas dans le buffer, écrit s'il y a lieu le buffer dans le fichier, puis lit la tranche du fichier qui contient le record en ajout. Mémoire écriture et ajout. Le pointeur du record courant est sur le record ajouté, donc sur le nouveau dernier record du fichier.

- **Entrée** : A = canal ; DE = pointeur en mémoire.
- **Sortie** : CC = exécution correcte ; CS = erreur, numéro dans A .
- **Modifie** : AF .
- **Erreurs** : 6 = end of file ; 22 = channel error ; 30 = device timeout ; 32 = write error ; 33 = read error.

?SETREC — positionne sur le record spécifié

- **Entrée** : A = canal ; HL = numéro du record.
- **Sortie** : CC = exécution correcte ; CS = erreur, numéro dans A .
- **Modifie** : AF .

- **Erreurs** : 17 = out of file ; 22 = channel error.

?GETREC — donne le numéro du record courant

- **Entrée** : A = canal.
- **Sortie** : HL = numéro du record courant ; CS = erreur, numéro dans A .
- **Modifie** : AF , HL .
- **Erreur** : 22 = channel error.

?POSREC — recherche un record par contenu

Recherche depuis le record courant le premier record qui contient la chaîne pointée par DE , de longueur B et commençant à C caractères du début du record. Le record trouvé devient le record courant. Dans le cas contraire on a le message d'erreur *end of file* et le record courant est le dernier record du fichier.

La recherche commence tout d'abord dans le buffer. Si l'on ne trouve pas dans le buffer, celui-ci est écrit dans le fichier s'il y a lieu, puis la recherche continue. Le processus se répète jusqu'à ce que la recherche aboutisse ou que l'on ait atteint la fin du fichier.

- **Entrée** : A = canal ; DE = pointeur argument de recherche ; B = longueur argument de recherche ; C = offset dans le record.
- **Sortie** : CC = exécution correcte ; CS = erreur, numéro dans A .
- **Modifie** : AF .
- **Erreurs** : 6 = end of file ; 22 = channel error ; 30 = device timeout ; 32 = write error ; 33 = read error ; 51 = illegal search parameters.

?RARGS — donne les paramètres du fichier ouvert de canal A

- **Entrée** : A = canal.
- **Sortie** : A = canal ; HL = nombre de records dans le fichier ; DE = nombre de records utilisés.
- **Modifie** : AF , HL , DE .
- **Erreur** : 22 = channel error.

Remarque : la soustraction directe en sortie d'appel $HL - DE$ donne dans HL le nombre de records libres.

?WARGS — change le nombre de records utilisés

- **Entrée** : DE = nombre de records utilisés.
- **Modifie** : AF .
- **Erreurs** : 17 = out of file ; 22 = channel error.

Remarque : cet appel permet de supprimer physiquement des enregistrements **à la fin du fichier**. Il permet également d'en créer **avec un contenu quelconque**.

Annexe — table des symboles $mat.st$ (décodée)

Structure d'une entrée $.st$ (8 octets, identique à $flo.st$) : **2 octets de valeur** (poids fort en tête) suivis du **nom sur 6 octets, inversé**, masqué à 7 bits, complété d'espaces. Le mécanisme d'appel est $RST\ 28H$: l'opcode $0xE7$ suivi d'un octet sélecteur ; donc *valeur du vecteur* = (*sélecteur* $\times$ 256) + $0xE7$. Exemple : $?FSIN = 0x91E7 \rightarrow .W$

Sélecteur	Symbole	Sélecteur	Symbole
0x80	?BINDEC (BINDE)	0x95	?FINVER (FINVE)
0x81	?DIBIDE (DIBID)	0x96	?FRANDOM (RANDO)
0x82	?WRBIDE (WRBID)	0x97	?FINT
0x83	?DECBIN (DECBI)	0x98	?FFRAC
0x84	?GETDEB (GEDEB)	0x99	?FCLR
0x85	?HLTBIN (HLTBI)	0x9A	?FPI
0x86	?BINTHL (BINTH)	0x9B	?FABS
0x87	?DECFOR (DECFO)	0x9C	?FCHSIG (FCHSI)
0x88	?FOPER	0x9D	?FRND
0x89	?FADD	0x9E	?FRND5
0x8A	?FSUB	0x9F	?FCOMP
0x8B	?FMUL	0xA0	?RCREAT (RCREA)
0x8C	?FDIV	0xA1	?ROPEN
0x8D	?FPOW	0xA2	?RCLOSE (RCLOS)
0x8E	?FSQRT	0xA3	?RRESET (RRESE)
0x8F	?FEXP	0xA4	?RDREC
0x90	?FLOG	0xA5	?WRREC
0x91	?FSIN	0xA6	?NEXREC (NEXRE)
0x92	?FCOS	0xA7	?ADDREC (ADDRE)
0x93	?FTAN	0xA8	?SETREC (SETRE)
0x94	?FATAN	0xA9	?GETREC (GETRE)
		0xAA	?POSREC (POSRE)
		0xAB	?RARGS
		0xAC	?WARGS
		0xAD	?PTREC

Les noms sont tronqués à 6 caractères dans `mat.st` (entre parenthèses la forme stockée quand elle diffère du nom complet du manuel). Entrées de service en tête de table : `VATABLE(X) = 0x2C00` (adresse de l'extension `TABEX`), `TEM = 0x00E7`, `SYS = 0x0100`.